CS 4122: Reinforcement Learning

Programming Assignment #4 (for Module 4)

Release date: 27th April, 2026, 1:30 am (IST)

Due date: 16th May, 2026, 11:59 pm (IST)

Maximum Score: 100 marks (this assignment is graded)

Read the following before you move forward:

1. This programming assignment is worth 10% of the total marks of this course.

2. Late policy: You can be late by a maximum of 3 days. Your assignment will not be accepted more than 3 days after the due date. The actual score and received score (received score is actual score minus penalty for late submission) are related as follows:

$$\text{received score} = \begin{cases} \text{actual score} & ;\text{late by 0 days} \\ 0.9 \cdot \text{actual score} & ;\text{late by 1 day} \\ 0.8 \cdot \text{actual score} & ;\text{late by 2 days} \\ 0.7 \cdot \text{actual score} & ;\text{late by 3 days} \\ 0 & ;\text{late by more than 3 days} \end{cases}$$

3. This is a team project. You have to do the project with the team that was finally selected for you. Only one submission per team. To submit, go to your team's Google drive folder. The Google drive link was already emailed to you. Then go to the sub-folder titled Programming Assignment 4 and drop the following SIX files (DON'T zip/compress these files, DON'T submit any other files):
   (a) report.pdf (b) car_dataset.csv (c) training.py (d) The final trained model 1 (named DQN_offline) (e) The final trained model 2 (named DQN_offline_true) (f) testing.py
   Deliverables are scattered in sections 4 to 6. Read them carefully to understand what needs to be submitted for each of the above six files.

4. The first page of report.pdf should contain a detailed description of the contribution made my individual team members. NOTE: Documenting the work is NOT a valid contribution.

5. Even though each team members may contribute to a specific section of the assignment, every team member should have the technical understanding of the entirety of the project.

6. Plagiarism, if detected, will lead to a score of ZERO for all the team members!
   a. It is ok to take help from AI tools. It is not acceptable to submit a code generated by AI without understanding every line of the code.
   b. It is ok to take help from other teams. But, it is a bad idea to keep open another teams work in front of you while you do the assignment. While you may think you are just taking help, your reports and your codes may get heavily influenced by the other team to the point that it will qualify as being plagiarized.

Double Deep Q-Learning for Mountain Car

In this programming assignment, you will be implementing double Deep Q-Learning for the Mountain Car environment of Gymnasium (Mountain Car - Gymnasium Documentation). Please do not confuse this with the mountain car continuous, also under the same category of Classic Control environments.

Section 1: Installation

If you are using Spyder that comes with Anaconda, execute the pip command in Anaconda prompt to install Gymnasium library and few other supporting libraries required for this assignment (for Colab, pip should be replaced with !pip):

pip install gymnasium

pip install swig

pip install gymnasium[classic-control]

pip install pygame

2nd line: swig is a library to interface C/C++ libraries with Python. 3rd line: classic-control package of Gymnasium contains the Mountain Car environment. 4th line: pygame is a library that is required for animation. Few things to note:

1. Install Gymnasium and NOT OpenAI Gym. Gym and Gymnasium are two different libraries that contains environments for reinforcement learning. Gymnasium is the newer version of Gym and supports more environments than Gym.
2. You need a deep learning library for this assignment. So, install either Keras, Tensorflow, or Pytorch. The YouTube video which I mentioned in section 2 uses Keras/Tensorflow.

Section 2: Sharing workload

Unfortunately, sharing workload in this assignment is quite tough. This is because there is only one main task which you will encounter in section 5, i.e. to write the code to train a DQN model for the mountain car environment. My sincere advice is that at least two team members should write their own code for section 5 INDEPENDENTLY. This is because depending on the coding style and the data structures used, the training speed of two different codes can differ significantly.

Section 3: Useful resources

1. To clear the fundamentals of Deep Q-Learning, check the slides of lectures 29 to 33. You will be implementing double Deep Q-Learning in this assignment whose pseudocode is there in the end of the lecture slides mentioned above.

2. In this programming assignment folder, there is a Python script *CartPole_DQN_training.py*. This Python script trains DQN architecture 2 for the cart pole environment of OpenAI Gymnasium using Keras/Tensorflow libraries. It trains DQN using replay buffer and target network, i.e. algorithm 3 in the slides of lectures 29 to 33. Pay attention to lines 47, 72, and 88 of this CartPole_DQN_training.py. Try to understand the significance of (1-terminated_batch) in line 88.

Section 4: Get familiar with the Mountain Car environment

There are two steps to get familiar with the mountain car environment of OpenAI Gym.

Step 1: Go through the environment's documentation to understand the environment:

Mountain Car - Gymnasium Documentation

Deliverables (6 marks): Answer the following questions about the mountain car environment. The answers should be there only in *report.pdf*. Your answers to the above questions MUST indicate that you read the webpage. Otherwise, you will get **zero** for that specific question.

1. How many actions are there in the action space? Very brief answer only.
2. Is it possible to accelerate to the left, and to the right at the same time? Justify your answer.
3. How many observations are there? Very brief answer only.
4. How many observations are there in the observation space? Very brief answer only. NOTE: Questions 3 and 4 are different.
5. What are the sources of randomness for this environment?
6. While training a DQN, there is a "convergence criteria" after which we should stop training. What should be this convergence criteria according to what you've read? The answer is open ended, however your answers must make logical sense.

Step 2: It's game time!!! You are given a Python script *MountainCar_PlayGame.py*. You can run this to play the game using keyboard. Don't start playing the game immediately! Read through step 2, complete the deliverables, and then play the game.

The idea of this step is three folds. *First,* to get you acquainted with the feature of Gymnasium that allows you to control an environment using keyboard. *Second,* to get you comfortable playing the game as it is required for step 3 where we will do some data collection. *Third,* to get you to appreciate the difficulty/ease of the game so that you can access the model that you will train in section 3.

> CAUTION: If you run this in Jupyter notebooks like that in Kaggle and Colab, it may flag an error and also the graphics may not appear. It should work in ipynb though.

When you run *MountainCar_PlayGame.py:*

1. A window titled "pygame window" appears where you can play the game. You have to use *a, s, and d* keys to play the game.

2. Unless you click the × sign on the right side of the "pygame window", episodes will run one after the other. Clicking the × sign is the correct way to stop the game. Get in the habit of clicking the × sign to stop the game rather than doing it abruptly in any other way. In case, you abruptly stopped the game, execute the code in line 38 to close the pygame window.
3. After every episode, the total reward will be displayed in the console.

Deliverables (4 marks): Consider line 31 of *MountainCar_PlayGame.py*. It uses a function named *gymnasium.utils.play.play*. The description of the function is given in the following webpage:

https://www.gymlibrary.dev/api/utils/

Based on your reading of the above webpage, answer the following questions. The answer to the above questions should be there ONLY in *report.pdf*.

1. Associate the keys **a**, **s**, and **d** with the respective actions (Accelerate to the left, don't accelerate, and accelerate to the right).
2. In line 31, what is the function of *noop*?
3. In line 31, what is the function of *callback=total_reward_func*?
4. Suppose the game is too fast for you, what should you change in line 31 and how?

Step 3: Recall that Deep Q-Learning is an off-policy algorithm. This means that we can use data collected from other resources, in this case from human actions, to train out Deep-Q Network (DQN). That is why it is important to some extent that you get comfortable playing the game before starting this step (this is not to be taken too seriously).

You are given a Python script titled *MountainCar_CollectOfflineData.py* and an empty csv file *car_dataset.csv*. The Python script is similar to that in Step 2. Just that it collects the data acquired during gameplay and saves it in the csv file which has the following columns (columns have their usual meaning):

Episode # **State** **Action** **Reward** **Next State** **Terminated**

Your task: Collect at least 30 episodes of the game in *car_dataset.csv*. Please keep the following points in mind while collecting the dataset:

1. You don't have to collect all 30 episodes at once. You can run *MountainCar_CollectOfflineData.py*, play a few episodes, and then close the pygame window. When you run the Python script again, the new data that is collected will get appended onto the existing dataset.
2. All the team mates can play the game and then concatenate the dataset. This will speed-up data collection. This will lead to heterogeneity in data which should help a learning algorithm.
3. It is highly suggested that you use × sign on the right side of the pygame window to stop the game play. In case you close it any other way, run lines 55 to 67 of the Python script to save the data.

Deliverables: Submit *car_dataset.csv* that contains the data collected during game play. No points for this step. But, if you don't submit *car_dataset.csv*, you will lose 10 points.

Section 5: Train DQN Model

In this section you will train TWO double DQN model; the first model should be without the data collected in previous section and the second one using the data. You must follow these rules:

1. The double DQN that you implement should use *algorithm 5* of lectures 29 to 33 (check the last two slides).
2. Mountain car environment is notoriously difficult to train because of sparse reward signal. To address the issue of sparse reward, many implementations uses reward shaping (reward shaping means changing the rewards received by the environment, for training the model). You must NOT use any form of reward shaping.
3. You CANNOT use existing Deep RL libraries like Keras RL, Stable Baselines, Tensorflow Agents etc. You will get ZERO for this section if you do so. You can use deep learning libraries though.

BRUTAL GRADING!!

Apart from the above rules, you must also follow all the rules in the following YouTube video:

https://youtu.be/sXodn-Gj_mo

If you fail to follow even one of the rules in the above YouTube video, will get ZERO for this section WITHOUT MERCY.

Deliverables (75 marks): You are given a Python script *training.py*. This script contains the bare basic skeleton of the DQN training code along with a function that loads the data collected in step 3 of section 4. You must NOT change the overall structure of the skeleton. There are two functions in *training.py*: *DQN_training* and *plot_reward*. Your task is to write the code for these two functions. Few instructions about *DQN_training*:

1. This function should implement a double Q-learning. The pseudocode for double DQN is given in algorithm 5 (not algorithm 4) of lectures 29 to 33. The output of the function is the final trained predict DQN model, and a Numpy array containing total reward per episode.
2. In function has an argument called *use_offline_data*. If this variable is *True* then the data collected in step 3 should be used for training; else not.
   - If *use_offline_data=False*, then it is business as usual, i.e. your code will be similar to that in the resources folder.
   - If *use_offline_data=True*, then we will initialize the replay buffer with the data collected offline. For the first E episodes, you will NOT append any data collected from the interaction with the environment onto the replay buffer. After E episodes, the data collected from the interaction with the environment should be appended to the replay buffer. The exact value of E is an hyperparameter. In many ways it depends on how good the data collected in step 3 is. If you got high total rewards in step 3, E can be high. In this regard note that one of the argument of the function *load_offline_data* is *min_score*. Only those episodes # will be loaded whose total reward is $\geq$ *min_score*. So, the higher the *min_score*, the quality of data increases but the

amount of data decreases. By default, *min_score* is set to $-\infty$ and hence all the episodes are loaded.

3. The final trained models should be saved and submitted. You should submit both models, using *use_offline_data=False* as well as with *use_offline_data=True*. Naturally name the model that does not use the offline data as "DQN_offline_false" and the one that does as "DQN_offline_true". The size of your models should not be greater than 2 MB each.

The function *plot_reward* should plot the following in the same graph: *(i)* total reward per episode, and *(ii)* moving average of the total reward. The plots for both *use_offline_data=False* and *use_offline_data=True* should be included in *report.pdf*.

The following points may also be of significant help:

1. For your own benefit, don't write two separate code for *use_offline_data=False* or *use_offline_data=True*. The difference between them is not much.
2. Don't forget to save the model periodically using an automated code. This is because your laptop, Kaggle/Colab notebooks can switch off (or go to sleep) if there is prolonged inactivity which is often the case when you are training a model for a long time. If you save your model, you can load it and start from where your progress stopped.
3. Don't used any complicated neural network model. It will take a lot of time to train it. A neural network model with 2-3 hidden layers and not more than 128 neurons per hidden layer is more than enough. In fact, 128 neurons is too much and so is the size of 2 MB mentioned move. The size of my model is less than 100 KB.
4. GPU will NOT increase the training speed significantly. This is one of the curses of Deep RL (unless you are using advanced techniques like multi-agent RL).
5. The model can take a significantly long time to start showing any improvement in performance. So, if you are not seeing any training progress it is possible that the code is correct and it is just taking a long time to train. A good idea is to keep running the code of the side and parallelly debugging a copy of the code.

Section 6: Testing the final model

Deliverables (15 marks): Finally, you will write a code to test the final model that you have trained. You are given a Python script *testing.py*. This script is supposed to show the animation of one complete episode of Mountain Car environment using the final model that you have trained in Section 5. *testing.py* contains clear instructions about writing the required code. Fill *testing.py* based on the instructions. You have to submit *testing.py*. I should be able to run and see the desired output in the click of a button.